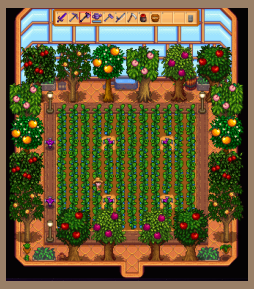
Implementação do Método Simplex em Julia

Aplicação: a estufa de Stardew Valley





Jhonatan B. L. Rodrigues – 199543

Sumário

1 Escopo **1**

2 O método simplex e o código **2**
2.1 Leitura dos dados e forma padrão 2
2.2 Base e solução básica 3
2.3 Multiplicadores e custos reduzidos 4
2.4 Teste de otimalidade 4
2.5 Variável que entra 5
2.6 Direção e problema ilimitado 5
2.7 Razão mínima e troca de base 5
2.8 Programa principal 6

3 Validação em um exemplo pequeno **6**

4 Aplicação: a estufa de Stardew Valley **7**
4.1 O jogo 7
4.2 O problema 9
4.3 Formulação 9
4.4 Passagem para a forma padrão (1) 10
4.5 Execução 11
4.6 Solução ótima 11

5 Limitações e melhorias **12**

Referências **13**

1 Escopo

O programa resolve

$$\min\ c^\top x \quad \text{sujeito a} \quad Ax = b, \quad x \geq 0, \tag{1}$$

com $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$ e $m \leq n$. Ele exige que o usuário informe uma *base inicial factível*; não há Fase I.

Como aplicação, o código é usado para decidir o que plantar e como processar a colheita na estufa do jogo *Stardew Valley* (Seção 4), um problema de programação linear pequeno, mas com estrutura rica e degenerada.

Roteiro do relatório

- Seção 2: o método simplex, explicado junto com o código em Julia.
- Seção 3: validação em um exemplo pequeno e em um problema ilimitado.
- Seção 4: o jogo, o modelo e a solução ótima da estufa.
- Seção 5: limitações e melhorias.

2 O método simplex e o código

O simplex percorre *vértices* do poliedro $\{x : Ax = b,\ x \geq 0\}$, passando de um vértice a outro vizinho sem piorar o objetivo, até que nenhum vizinho melhore. Cada vértice corresponde a uma *base*, e cada passagem entre vértices é uma volta do laço `while`.

2.1 Leitura dos dados e forma padrão

```
1
2  function ler_dados_simplex()
3      print("Insira a quantidade de restricoes e variaveis (m n): ")
4      m, n = parse.(Int, split(readline()))
5
6      if m <= 0 || n < m
7          throw(ArgumentError("Deve valer 0 < m <= n."))
8      end
9
10     println("Insira os $n coeficientes do vetor de custos:")
11     c = parse.(Float64, split(readline()))
12
13     println("Insira os $m valores do vetor de recursos:")
14     b = parse.(Float64, split(readline()))
15
16     A = Matrix{Float64}(undef, m, n)
17
18     println("Insira os coeficientes da matriz A ($m x $n), linha por linha:")
19     for i in 1:m
20         A[i, :] = parse.(Float64, split(readline()))
21     end
22
23     println("Insira os $m indices das variaveis basicas:")
24     B = parse.(Int, split(readline()))
25
26     if length(c) != n || length(b) != m
27         throw(ArgumentError("Dimensoes incorretas dos vetores c ou b."))
28     end
29
30     if size(A) != (m, n)
31         throw(ArgumentError("Dimensoes incorretas da matriz A."))
32     end
33
34     if length(B) != m || length(unique(B)) != m ||
35        any(j -> j < 1 || j > n, B)
36         throw(ArgumentError("Base invalida: quantidade, indices ou repeticoes."))
37     end
38
39     return m, n, c, b, A, B
40 end
```

Método. O código trabalha com o problema na forma padrão (1). A restrição $Ax = b$ tem m equações e n incógnitas; uma base escolhe m colunas de A linearmente independentes, e por isso é necessário $m \leq n$. Para chegar a essa forma partindo de um problema qualquer, aplicam-se três regras: maximizar Z equivale a minimizar $-Z$; cada desigualdade $\leq$ ganha uma variável de folga não negativa; e todos os termos com variáveis ficam

do lado esquerdo.

Código. A leitura é interativa pela entrada padrão. Cada item ocupa uma linha, com números separados por espaço, lidos por `parse.(Tipo, split(readline()))`:

Linha	Conteúdo	Leitura no código
1	m n	l. 3
2	n custos	l. 7
3	m recursos b	l. 9
4 a 3+m	linhas da matriz A (n números cada)	l. 26–l. 29
4+m	m índices da base inicial (a partir de 1)	l. 31

Tabela 1. Estrutura do arquivo de entrada.

As validações são:

- l. 9–l. 11: exige $0 < m \leq n$, a condição acima;
- l. 35–l. 37: confere que a base tem m índices, sem repetição (`unique`) e dentro de $1, \ldots, n$.

A validação confere só a *forma* dos dados: não verifica se as colunas da base são independentes nem se a base é factível; isso aparece nas próximas etapas.

2.2 Base e solução básica

```
45          max_iter = 1000
46          iter = 0
47
48          while inter < max_iter
49          N = setdiff(1:n, B)
50          AB = A[:, B]
51
52
53          # Solucao basica
54          xB = AB \ b
55
56          if any(xB .< -tol)
57              error("A base inicial nao e factivel.")
58          end
```

Método

Seja B um conjunto de m índices tal que a submatriz A_B (matriz básica) é inversível, e seja N (índices da parte não básica) o complemento. Particionando $x = (x_B, x_N)$, a *solução básica* associada a B é

$$x_N = 0, \qquad x_B = A_B^{-1}\, b. \tag{2}$$

Ela é *factível* se $x_B \geq 0$, e seu objetivo é $z = c_B^{\top} x_B$. Soluções básicas factíveis são exatamente os vértices do poliedro.

Código. l. 45 define a tolerância numérica `tol = 1e-9`, usada em todos os testes de sinal para absorver erros de ponto flutuante. A cada volta, l. 48 recalcula N e l. 49 extrai A_B com `A[:, B]`. l. 53 calcula x_B com `AB \ b`: o operador `\` resolve o sistema linear por fatoração, o que é mais estável do que formar A_B^{-1}. l. 54 para com erro se alguma componente de x_B for menor que $-$`tol`, isto é, se a base não for factível. Depois da primeira volta, essa checagem nunca dispara, pois a base só muda por pivoteamentos que preservam a viabilidade.

2.3 Multiplicadores e custos reduzidos

```
60          # Multiplicadores simplex
61          lambda = AB' \ c[B]
```

Método

Os *multiplicadores simplex* da base B são

$$\lambda = A_B^{-\top} c_B, \tag{3}$$

e o *custo reduzido* de cada variável não básica $j \in N$ é

$$\bar{c}_j = c_j - \lambda^\top A_j. \tag{4}$$

Se x_j cresce de $\theta \geq 0$ e as básicas se reajustam para manter $Ax = b$, o objetivo varia de $\bar{c}_j\,\theta$. Portanto $\bar{c}_j < 0$ indica que *entrar* com x_j melhora (diminui) o objetivo.

Código. l. 59 resolve $A_B^\top \lambda = c_B$ com `AB' \ c[B]`, onde `AB'` é a transposta. l. 62 calcula todos os custos reduzidos de uma vez: `A[:, N]' * lambda` é o vetor dos produtos $\lambda^\top A_j$ para $j \in N$, e `c[N] -` subtrai esse vetor de c_N. O resultado `custos` tem uma entrada por variável não básica, na ordem de N.

2.4 Teste de otimalidade

```
63          # Custos reduzidos das variaveis nao basicas
64
65          custos = c[N] - A[:, N]' * lambda
66
67          # Teste de otimalidade
68          k = argmin(custos)
69
70          if custos[k] >= -tol
71              return 1, B
72          end
```

Proposição 1 (Otimalidade)**.** *Se $\bar{c}_j \geq 0$ para todo $j \in N$, a solução básica factível corrente é ótima para* (1)*.*

A l. 65 acha a posição k do menor custo reduzido. Se esse mínimo é maior ou igual a $-$`tol` (l. 70), nenhum $\bar{c}_j$ é negativo e a Proposição 1 garante o ótimo: a função devolve o código `1` e a base (l. 68).

2.5 Variável que entra

```
74          # Variavel que entra na base
75          entra = N[k]
```

Entre as variáveis com $\bar{c}_j < 0$, é preciso escolher uma. O código usa a regra de Dantzig: a de custo reduzido mais negativo, $j^* = \arg\min_{j \in N} \bar{c}_j$, que é a de maior queda por unidade. `k` vem de l. 65 e indexa o vetor `custos`, que segue a ordem de `N`; por isso l. 75 converte para o índice real da variável, `entra = N[k]`.

2.6 Direção e problema ilimitado

```
77          # Direcao simplex
78          d = AB \ A[:, entra]
79
80          # Teste de ilimitacao
81          indices = findall(v -> v > tol, d)
82
83          if isempty(indices)
84              return -1, B
85          end
```

Método

Ao aumentar x_j ($j =$ `entra`), as básicas passam a ser $x_B(\theta) = x_B - \theta d$, com a *direção simplex*

$$d = A_B^{-1} A_j. \tag{5}$$

Se $d \leq 0$, nenhuma básica diminui e θ cresce sem limite; como $\bar{c}_j < 0$, o objetivo cai sem cota: o problema é *ilimitado*.

Código. l. 75 calcula d com `AB \ A[:, entra]`. l. 78 guarda em `indices` as posições onde $d_i >$ `tol`, as únicas que limitam o passo. Se `indices` está vazio (l. 83), vale $d \leq 0$ e a função devolve `-1` (l. 84).

2.7 Razão mínima e troca de base

```
87          # Teste da razao minima
88          razoes = xB[indices] ./ d[indices]
89          pos = argmin(razoes)
90          sai_pos = indices[pos]
91
92          # Atualiza a base na posicao da variavel que sai
93          B[sai_pos] = entra
94                  iter =  iter + 1
95
96      end
```

Método

Caso contrário, a primeira básica a zerar determina o passo máximo

$$\theta^* = \min_{i:\, d_i > 0} \frac{(x_B)_i}{d_i}. \tag{6}$$

A variável básica da posição que atinge o mínimo sai da base e x_j ocupa o lugar dela. Esse passo mantém $x_B(\theta^*) \geq 0$, e por isso a nova base continua factível. O novo objetivo é $z + \bar{c}_j\, \theta^* \leq z$.

Código. l. 88 calcula as razões só nas posições de `indices`. `argmin(razoes)` (l. 89) devolve a posição do mínimo *dentro* de `razoes`, e l. 90 a converte para a posição em B com `indices[pos]`. Se houver empate, vale a primeira ocorrência. l. 93 faz a troca, `B[sai_pos] = entra`, e l. 96 fecha o `while`: a execução volta a l. 48 com a nova base. Observe que a função altera o vetor `B` recebido.

Degeneração. Quando alguma componente de x_B é zero, θ^* pode ser zero: a base muda, mas x e z não. Esse pivoteamento é dito *degenerado*. Sem degeneração, cada volta reduz z estritamente e o método termina em número finito de passos.

2.8 Programa principal

```
100 m, n, c, b, A, B = ler_dados_simplex()
101
102 status, B = simplex(m, n, c, b, A, B)
103
104 if status == 1
105     x = zeros(Float64, n)
106     x[B] = A[:, B] \ b
107     println("Solucao otima encontrada.")
108     println("Vetor de variaveis: ", x)
109     println("Variaveis basicas: ", B)
110     println("Valor otimo: ", c' * x)
111 elseif status == -1
112     println("Problema ilimitado.")
```

l. 100 lê os dados e l. 102 chama o simplex. Se o código de retorno é 1, reconstrói-se a solução completa: zeros nas não básicas e $x_B = A_B^{-1} b$, a mesma fórmula (2). l. 102–l. 105 imprimem o vetor, os índices da base e o valor ótimo $c^\top x$. Se o retorno é -1, informa que o problema é ilimitado.

3 Validação em um exemplo pequeno

Antes do modelo principal, o programa foi testado em

$$\max\ 3x + 5y \quad \text{s.a.} \quad x \leq 4,\ \ 2y \leq 12,\ \ 3x + 2y \leq 18,\ \ x, y \geq 0.$$

Com folgas x_3, x_4, x_5 e custos negados (o código minimiza), tem-se $c = (-3, -5, 0, 0, 0)$, $b = (4, 12, 18)$ e base inicial $\{3, 4, 5\}$. O arquivo de entrada, no formato da tabela acima, é:

```
3 5
-3 -5 0 0 0
4 12 18
1 0 1 0 0
0 2 0 1 0
3 2 0 0 1
3 4 5
```

Iter.	Base	z	Entra	Sai	θ^*
0	$\{3,4,5\}$	0	x_2 ($\bar{c}=-5$)	x_4	6
1	$\{3,2,5\}$	-30	x_1 ($\bar{c}=-3$)	x_5	2
2	$\{3,2,1\}$	-36	$\min \bar{c} = 1 \geq 0$: ótimo		

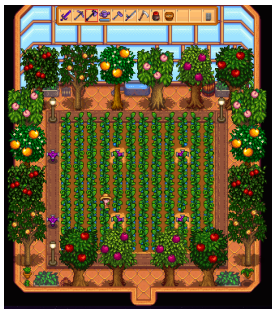
Tabela 2. Iterações do exemplo pequeno.

A solução é $x = 2$, $y = 6$, com lucro 36 (valor ótimo -36 na minimização), em 2 pivoteamentos. Um problema ilimitado de teste, $\min -x_1 - x_2$ com $x_1 - x_2 + x_3 = 1$ e base $\{3\}$, devolveu o código -1, como esperado: x_1 entra primeiro ($d = 1$, $\theta^* = 1$) e, na base $\{1\}$, x_2 tem $\bar{c} = -2$ e direção $d = -1 \leq 0$, sem limite para o crescimento.

4 Aplicação: a estufa de Stardew Valley

4.1 O jogo

Stardew Valley (ConcernedApe, 2016) é um jogo de simulação de fazenda em que o jogador herda uma propriedade no interior, planta, cria animais, pesca, minera e se relaciona com os moradores da cidade. O tempo é dividido em quatro estações de 28 dias, de modo que **um ano de jogo tem** $4 \times 28 = 112$ **dias**. Como o dinheiro (em *ouros*) limita praticamente tudo o que se faz, otimizar a renda da fazenda é uma questão natural, e um bom candidato a programação linear.



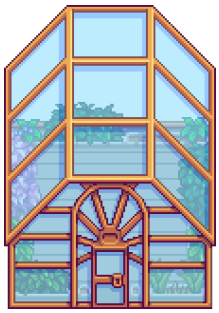


Figura 1. A estufa *(Imagens: Stardew Valley Wiki, © ConcernedApe.)*

A estufa. A estufa é uma construção que, depois de reformada, oferece **116 espaços de plantio** e, ao contrário do campo aberto, **não depende da estação**: qualquer cultura cresce o ano inteiro. Isso transforma o plantio em um problema de alocação, pois a única restrição é o espaço.

As duas culturas. O modelo compara duas frutas de alto valor:

- **Fruta Antiga** (*Ancient Fruit*): demora para dar o primeiro fruto, mas depois *rebrota* sozinha a cada poucos dias, sem precisar replantar. Não há gasto anual com sementes (assume-se que o jogador já as tem).
- **Carambola** (*Starfruit*): cresce mais rápido, mas é colhida uma vez e precisa ser replantada. Suas sementes custam ouro a cada replantio, o que gera o custo de 3 200 ouros por espaço ao longo do ano.



Fruta Antiga



Carambola

Figura 2. As duas culturas comparadas no modelo.

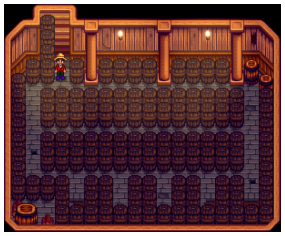
Três destinos para a colheita. Cada fruta colhida pode ser:

1. **vendida diretamente**, com qualidade normal;
2. **transformada em vinho comum** em um *barril de produção* (*keg*), processo que leva cerca de 7 dias;
3. **transformada em vinho e envelhecida** em um *barril de envelhecimento* (*cask*), instalado na adega, até atingir a qualidade *irídio*, a mais alta (cerca de 56 dias de envelhecimento).

A profissão **Artesão** (*Artisan*) aumenta em 40% o valor de bens artesanais como o vinho, e é esse o cenário adotado. O vinho vale o triplo da fruta, e o envelhecimento até irídio dobra o valor do vinho comum.



Barril de produção (*keg*)



Barril de envelhecimento (*cask*)

Figura 3. Equipamentos de processamento.

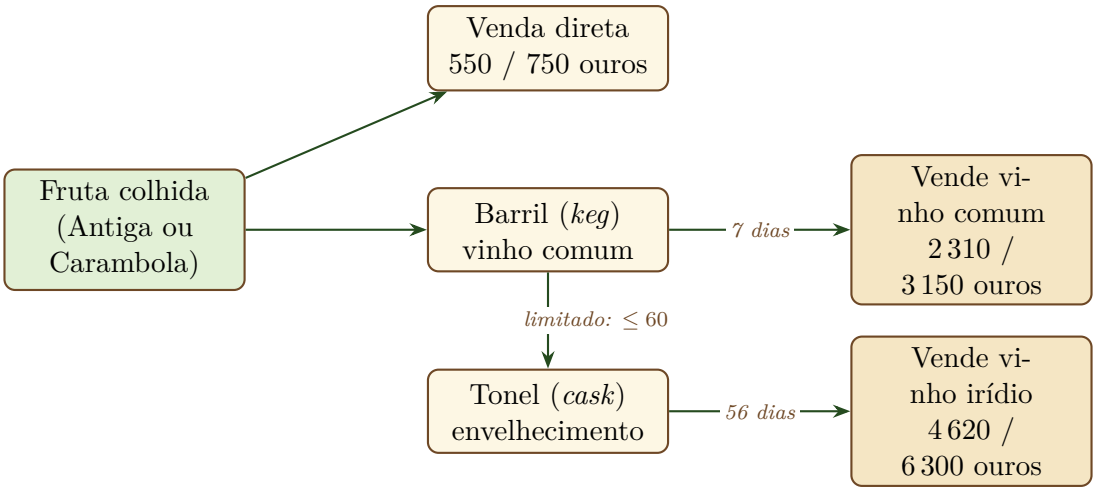


Figura 4. Caminhos possíveis para cada fruta colhida (valores para Fruta Antiga / Carambola, com Artesão).

4.2 O problema

Com 116 espaços de plantio e um ano de jogo (112 dias), deseja-se decidir entre vender as frutas diretamente, transformá-las em vinho comum ou envelhecer o vinho até a qualidade irídio, comparando Fruta Antiga e Carambola. Os equipamentos são 20 barris de produção (*kegs*) e 30 barris de envelhecimento (*casks*, o máximo que cabe na adega).

Parâmetro	Fruta Antiga	Carambola
Colheitas por espaço em 112 dias	12	8
Custo anual de sementes por espaço (ouros)	0	3 200
Venda direta, qualidade normal (ouros)	550	750
Vinho comum, com Artesão (ouros)	2 310	3 150
Vinho irídio, com Artesão (ouros)	4 620	6 300

Tabela 3. Parâmetros do modelo.

A capacidade dos kegs é $20\lfloor 112/7 \rfloor = 320$ vinhos por ano, e a dos casks é $30\lfloor 112/56 \rfloor = 60$ vinhos envelhecidos. Note a coerência dos preços: $550 \cdot 3 \cdot 1{,}4 = 2\,310$ e $2\,310 \cdot 2 = 4\,620$; o mesmo vale para a Carambola.

4.3 Formulação

Sejam d_A, d_C as frutas vendidas diretamente, x_A, x_C os vinhos comuns, y_A, y_C os vinhos envelhecidos, e n_A, n_C os espaços plantados com cada fruta (as letras A e C indicam Fruta

Antiga e Carambola). O problema é

$$
\begin{aligned}
\max\ Z = {} & 550\,d_A + 750\,d_C + 2310\,x_A + 3150\,x_C \\
& + 4620\,y_A + 6300\,y_C - 3200\,n_C \\
\text{s.a. } & x_A + x_C + y_A + y_C \le 320 && \text{(kegs)} \\
& y_A + y_C \le 60 && \text{(casks)} \\
& d_A + x_A + y_A \le 12\,n_A && \text{(frutas Antigas)} \\
& d_C + x_C + y_C \le 8\,n_C && \text{(Carambolas)} \\
& n_A + n_C \le 116 && \text{(área)}
\end{aligned}
$$

com todas as variáveis não negativas. Trata-se da relaxação contínua, pois plantas e frutas são inteiras no jogo.

4.4 Passagem para a forma padrão (1)

Aplicam-se as regras da Seção 2.1: minimizar $-Z$, somar uma folga em cada desigualdade e passar $12\,n_A$ e $8\,n_C$ para o lado esquerdo. O resultado tem $n = 13$ variáveis e $m = 5$ restrições.

Índice	Variável	Significado	c_j
x_1	d_A	Fruta Antiga vendida direto	-550
x_2	d_C	Carambola vendida direto	-750
x_3	x_A	vinho comum de Fruta Antiga	-2310
x_4	x_C	vinho comum de Carambola	-3150
x_5	y_A	vinho de Fruta Antiga envelhecido	-4620
x_6	y_C	vinho de Carambola envelhecido	-6300
x_7	n_A	espaços com Fruta Antiga	0
x_8	n_C	espaços com Carambola	$+3200$
$x_9, \ldots, x_{13}$	$s_1, \ldots, s_5$	folgas	0

Tabela 4. Variáveis e custos na forma padrão.

$$
A = \left[\begin{array}{cccccccc|ccccc}
0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
1 & 0 & 1 & 0 & 1 & 0 & -12 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 1 & 0 & 1 & 0 & 1 & 0 & -8 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1
\end{array}\right], \qquad b = \begin{bmatrix} 320 \\ 60 \\ 0 \\ 0 \\ 116 \end{bmatrix}.
$$

As colunas à direita da barra vertical formam I_5, a identidade das folgas. Por isso a base inicial $B = \{9, 10, 11, 12, 13\}$ é factível: $x_B = b \ge 0$, correspondendo a "não plantar nada". As linhas 3 e 4 têm $b = 0$, o que torna a base inicial degenerada. O arquivo de entrada, no formato da Seção 2.1, é:

```
5 13
-550 -750 -2310 -3150 -4620 -6300 0 3200 0 0 0 0 0
320 60 0 0 116
0 0 1 1 1 1 0 0 1 0 0 0 0
```

```
0 0 0 0 1 1 0 0 0 1 0 0 0
1 0 1 0 1 0 -12 0 0 0 1 0 0
0 1 0 1 0 1 0 -8 0 0 0 1 0
0 0 0 0 0 0 1 1 0 0 0 0 1
9 10 11 12 13
```

4.5 Execução

A Tabela 5 mostra o caminho do simplex. Em cada iteração, a variável que entra é a de custo reduzido mais negativo e a que sai vem da razão mínima. O valor z é o da solução no início da iteração, com o sinal da minimização.

Iter.	z	$\min \bar{c}_j$	Entra	Sai	θ^*
0	0	$-6\,300$	x_6 (y_C)	x_{12} (s_4)	0
1	0	$-47\,200$	x_8 (n_C)	x_{10} (s_2)	7,5
2	$-354\,000$	$-2\,750$	x_4 (x_C)	x_9 (s_1)	260
3	$-1\,069\,000$	-550	x_1 (d_A)	x_{11} (s_3)	0
4	$-1\,069\,000$	$-6\,600$	x_7 (n_A)	x_{13} (s_5)	76
5	$-1\,570\,600$	$+165$	todos $\bar{c}_j \geq 0$: ótimo		

Tabela 5. Iterações do simplex no problema da estufa.

As iterações 0 e 3 são degeneradas ($\theta^* = 0$): a base muda sem alterar z, porque as linhas de frutas têm $b = 0$. A regra de Dantzig atravessou ambas sem ciclar. No total foram 5 pivoteamentos.

4.6 Solução ótima

A base final é $B = \{4, 8, 1, 6, 7\}$, e o vetor ótimo é

$$x^* = (\underbrace{912}_{d_A},\ 0,\ 0,\ \underbrace{260}_{x_C},\ 0,\ \underbrace{60}_{y_C},\ \underbrace{76}_{n_A},\ \underbrace{40}_{n_C},\ 0, 0, 0, 0, 0),$$

com $c^\top x^* = -1\,570\,600$, isto é, lucro $Z^* = 1\,570\,600$ ouros. Conferindo termo a termo:

$$912 \cdot 550 + 260 \cdot 3150 + 60 \cdot 6300 - 40 \cdot 3200 = 501\,600 + 819\,000 + 378\,000 - 128\,000 = 1\,570\,600.$$

Todas as folgas são nulas: os 116 espaços, os 20 kegs ($260 + 60 = 320$ vinhos), os 30 casks (60 vinhos) e todas as frutas são usados por inteiro. As $40 \cdot 8 = 320$ Carambolas colhidas viram exatamente os 320 vinhos; as $76 \cdot 12 = 912$ Frutas Antigas são vendidas diretamente. A solução é inteira, então não há perda por usar a relaxação contínua neste cenário.

Em linguagem de jogador

Plante **76 espaços de Fruta Antiga** e venda todas as frutas sem processar. Plante **40 espaços de Carambola**, transforme todas em vinho nos 20 kegs e envelheça 60 desses vinhos nos 30 casks; os 260 restantes são vendidos como vinho comum. A lógica é que os equipamentos são escassos e valem mais para a Carambola, cujo vinho

rende mais por unidade, enquanto a Fruta Antiga, sem custo de sementes, ocupa o espaço que sobra.

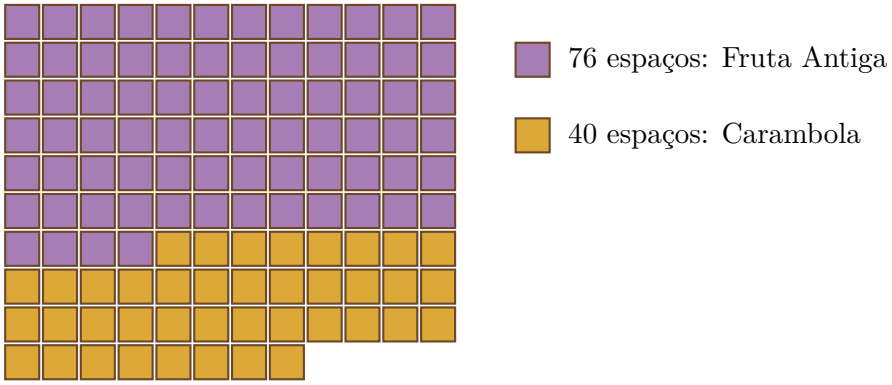


Figura 5. Mapa esquemático dos 116 espaços da estufa na solução ótima.

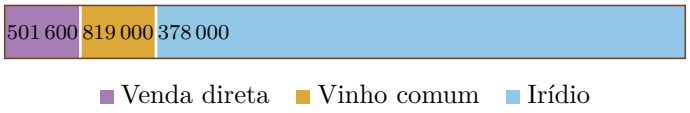


Figura 6. Receita bruta de 1 698 600 ouros por origem; descontados 128 000 em sementes, o lucro é 1 570 600.

5 Limitações e melhorias

- **Sem Fase I.** A base inicial precisa ser factível. Com restrições $\geq$ ou $=$, seria preciso um problema auxiliar para achar a base.
- **Ciclagem.** A regra de Dantzig pode ciclar em casos degenerados.
- **Informação limitada.** O código só distingue ótimo e ilimitado; não relata infactibilidade nem múltiplos ótimos.
- **Base singular.** Se A_B for singular, `AB \ b` falha ou produz valores inválidos; a validação de entrada não detecta isso.
- **Hipóteses do modelo.** Replantio imediato, 112 dias, kegs e casks prontos desde o início, qualidade normal de venda direta e sementes de Fruta Antiga já disponíveis.

Stardew Valley © ConcernedApe. Imagens do jogo usadas apenas para fins ilustrativos e acadêmicos.

Referências

[1] D. G. Luenberger e Y. Ye. *Linear and Nonlinear Programming*. 4.ª ed. Springer, 2016. https://doi.org/10.1007/978-3-319-18842-3

[2] G. B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, 1963.

[3] Stardew Valley Wiki. *Stardew Valley Wiki* [página inicial]. https://stardewvalleywiki.com/Stardew_Valley_Wiki. Acesso em 26 out. 2026.